



IIC2650: Fundamentos de Lenguajes de Computación

Tarea 2: Subprograms y OOP

Viernes 1-Mayo-2026

1 Objetivos

En este curso has aprendido todo lo importante para utilizar e implementar lenguajes, ahora es tu turno de usar distintas técnicas para ayudar a la Granja DCC.

1. **Importante:** La evaluación será hecha en base a *test cases*, cada parte de la tarea vale 50%.
2. Manejarás **Funciones de Primer Grado**. Es decir que construirás, entregarás y recibirás funciones dentro de tus funciones. Además, implementarás la interpretación de un lenguaje OOP.
3. **Fecha de entrega:** Martes 26 de Mayo
4. Esta tarea cuenta con política de atraso hasta 2 días.

Contenidos

1	Objetivos	1
2	Ejercicios	2
2.1	C++ (50%)	2
2.1.1	Puntaje	2
2.1.2	Map, Filter, Fold	2
2.1.3	Predicados combinables	2
2.1.4	Lazy Streams	3
2.1.5	Stable API	3
2.1.6	Matcha latte	4
2.1.7	Funciones definidas con funciones	5
2.1.8	Currying y partial application	5
2.1.9	Ordenamiento	5
2.1.10	Subscripciones	6
2.2	Ruby (50%)	6
2.2.1	Programación Orientada a Animales	6
3	Entrega	7
3.1	Classroom	7
3.2	Evaluación	8
3.3	Política de Atraso	8
4	Recomendaciones	8
4.1	Recomendaciones generales de parte de los ayudantes	8
4.2	Recursos	9
4.2.1	C++	9
4.2.2	Ruby	9
5	Uso de inteligencia Artificial	9

2 Ejercicios

2.1 C++ (50%)

C++ es un lenguaje de programación diseñado en 1979 por Bjarne Stroustrup. La intención de su creación fue extender al lenguaje de programación C y añadir mecanismos que permiten la manipulación de objetos. En ese sentido, desde el punto de vista de los lenguajes orientados a objetos, C++ es un lenguaje híbrido.

Para este curso utilizaremos el compilador *g++* con *std23* y compilaremos con el comando *make test*.



2.1.1 Puntaje

Se evaluarán **7 de los 9** ejercicios. Puedes ignorar 2 ejercicios que no desees realizar, si entregas más de 7 ejercicios entonces descontaremos los que tengan mayor puntaje.

2.1.2 Map, Filter, Fold

Hay tantos animales y cosechas en la granja que ir revisando uno por uno toma tiempo, por lo que decides definir tus propias funciones para que lo hagan por ti.

Define la función `farm_map(lst, f)` tal que retorne un vector equivalente a aplicar la función `f` a cada item de `lst`. **No está permitido usar `for` y `while`, pero sí recursión o funciones lambda. Además no puedes usar las versiones *built-in* equivalentes (`std::transform`, `std::copy_if`, `std::reduce`, etc).** Esta restricción solamente aplica en este ejercicio

```
vector<string> animals = {"vaca", "pollo", "oveja"};
function<string(const string&)> to_upper = [](const string &s) {
    string r = s;
    for (char &c : r) c = toupper(static_cast<unsigned char>(c));
    return r;
};
auto res = farm_map(animals, to_upper); // >>> {"VACA", "POLLO", "OVEJA"}
```

Define la función `farm_filter(lst, f)` tal que retorne un vector con aquellos items que retornen `true` al aplicar `f`.

```
vector<string> crops = {"maiz", "trigo", "choclo", "arroz"};
function<bool(const string&)> is_long = [](const string &s){ return s.size() > 4; };
auto filtered = farm_filter(crops, is_long); // >>> {"trigo", "choclo", "arroz"}
```

Define las funciones `farm_foldl(lst, init, f)` y `farm_foldr(lst, init, f)` tal que retornen el resultado de aplicar `f` acumulativamente a los items de `lst`. Puede asumir que el comportamiento de `foldl` es similar a la función *reduce* en python.

```
vector<int> nums = {1, 2, 3, 4, 5};
function<int(int, int)> sum = [](int acc, int x) { return acc + x; };
function<int(int, int)> sub = [](int acc, int x) { return acc - x; };
int l = farm_foldl(nums, 0, sum); // >>> 15
int r = farm_foldr(nums, 0, sub); // >>> -15
```

2.1.3 Predicados combinables

Define la función `andP(p, q)` tal que retorne un predicado equivalente a `p(x) && q(x)`.

```
function<bool(const int&)> is_even = [](const int& n) { return n % 2 == 0; };
function<bool(const int&)> is_pos = [](const int& n) { return n > 0; };
andP(is_even, is_pos)(4); // >>> true
```

Define la función `orP(p, q)` tal que retorne un predicado equivalente a $p(x) \vee q(x)$.

```
orP(is_even, is_pos)(-3); // >>> false
orP(is_even, is_pos)(-2); // >>> true
```

Define la función `notP(p)` tal que retorne un predicado equivalente a $\neg p(x)$.

```
notP(is_even)(3); // >>> true
```

2.1.4 Lazy Streams

Define streams infinitos usando funciones *thunks*. Un stream es una secuencia potencialmente infinita representada como una función que genera el siguiente elemento bajo demanda.

from(n) Define la función `from(n)` que genere un stream infinito comenzando en `n`.

```
auto stream = from(1);
// El stream representa: 1, 2, 3, 4, ...
```

lazy_map(stream, f) Define la función `lazy_map(stream, f)` que aplique `f` a cada elemento del stream sin evaluar todo.

```
auto squares = lazy_map(from(1), [](int n) { return n * n; });
// El stream representa: 1, 4, 9, 16, ...
```

lazy_filter(stream, pred) Define la función `lazy_filter(stream, pred)` que filtre elementos del stream según `pred`.

```
auto evens = lazy_filter(from(1), [](int n) { return n % 2 == 0; });
// El stream representa: 2, 4, 6, 8, ...
```

take(stream, k) Define la función `take(stream, k)` que extraiga los primeros `k` elementos del stream como un vector.

```
take(evens, 5); // >>> vector<int>{2, 4, 6, 8, 10}
take(squares, 3); // >>> vector<int>{1, 4, 9}
```

Utiliza estas funciones para definir los siguientes streams:

1. Cuadrados pares (4, 16, 36...)
2. Números primos (2, 3, 5, 7, 11...)
3. La serie Fibonacci (1, 1, 2, 3, 5, 8, 13...)

2.1.5 Stable API

Implementarás una API de consultas para los establos, deben incluir las siguientes operaciones:

1. `Query` es la clase para construir consultas a partir de un vector. `to_list()` resuelve la consulta a un vector.

```
Query<Establo>(establos).to_list(); // >>> vector<Establo>{...}
```

2. `where(cond)` retorna una consulta a partir de otra donde todos sus elementos `element` deben cumplir con `cond(element) == true`

```
Query<Establo>(establos)
  .where([](const Establo& e) { return e.capacidad >= 10; })
  .to_list(); // >>> vector<Establo>{...}
```

3. `select(f)` retorna una consulta donde sus elementos son el resultado de aplicar `f` a la consulta anterior.

```
Query<Establo>(establos)
  .select([](const Establo& e) { return e.nombre; })
  .to_list(); // >>> vector<string>{"establo_1", "establo_2", "establo_3"}
```

4. `order_by(cmp)` retorna una consulta a partir de otra donde sus elementos x, y cumplen que $x < y$ si y solo si $cmp(x, y) = true$

```
Query<Establo>(establos)
  .where([](const Establo& e) { return e.capacidad >= 10; })
  .select([](const Establo& e) { return pair<string, int>{e.nombre, e.capacidad}; })
  .order_by([](const auto& x, const auto& y) {
    return x.second == y.second ? x.first < y.first : x.second > y.second;
  })
  .to_list();
// >>> {"establo_2", 20}, {"establo_1", 12}
```

5. Debe ser posible obtener más consultas a partir de otras.

```
Query<Establo>(establos)
  .where([](const Establo& e) { return e.sector == "norte"; })
  .select([](const Establo& e) { return pair<string, string>{e.nombre, e.sector}; })
  .order_by([](const auto& x, const auto& y) { return x.first < y.first; })
  .to_list();
// >>> {"establo_1", "norte"}, {"establo_3", "norte"}
```

2.1.6 Matcha latte

Implementarás funciones de matching con strings basada en *lambdas*. Cada función matcher recibe una entrada y retorna `true` solo si el patrón hace match; en caso contrario, retorna `false`.

1. `match_string(s)`: retorna un matcher que verifica si la entrada es igual a `s`.

```
match_string("matcha latte")("matcha latte"); // >>> true
match_string("matcha mocha")("matcha latte"); // >>> false
```

2. `match_concat(p, q)`: retorna el matcher equivalente a matchear `p` seguido de `q`.

```
match_concat(match_string("matcha"), match_string(" latte"))("matcha latte"); // >>> true
```

3. `match_choice(p, q)`: retorna el matcher que evalúa si `p` o `q` hace match (en un OR lógico).

```
auto p = match_string("mocha");
auto q = match_string("espresso");
match_choice(p, q)("espresso"); // >>> true
match_choice(p, q)("mocha");    // >>> true
match_choice(p, q)("macchiato"); // >>> false
```

4. `match_many(p)`: hace match de `p` cero o más veces.

```
match_many(match_string("a"))("aaa"); // >>> true
match_many(match_string("a"))(""); // >>> true
match_many(match_string("a"))("aaab"); // >>> false
```

5. `match_any()`: retornar un matcher que hace match con cualquier carácter.

```
match_any()( "a" ); // >>> true
match_any()( "aa" ); // >>> false
```

6. Debe ser posible combinar matchers

```
match_many(match_any())("aaaaaaaaaaaaaaaaaaaaa"); // >>> true
match_concat(
    match_string("matcha "),
    match_choice(match_string("latte"), match_string("mocha"))
)("matcha latte"); // >>> true
```

2.1.7 Funciones definidas con funciones

Define la función `compose(f, g)` tal que `compose(f,g)(x)` sea equivalente a `f(g(x))`.

```
function<int(const int&)> f = [](const int& x) { return x + 1; };
function<int(const int&)> g = [](const int& x) { return x * 2; };
auto h = compose<int, int, int>(f, g);
h(3); // >>> 7
```

Define la función `pipe(1st)` tal que retorne la composición secuencial de **la lista de funciones**.

```
auto normalize = pipe<string>({
    [](string s) { s.erase(0, s.find_first_not_of(" \t\n\r")); s.erase(s.find_last_not_of(" \t\n\r") + 1);
    return s; },
    [](string s) { for (char &c : s) c = tolower(static_cast<unsigned char>(c)); return s; }
});

// farm_filter(farm_map({" HoLa ", " ", "Mundo "}, normalize), [](const string& s){ return !s.empty();
// })
// >>> {"hola", "mundo"}
```

2.1.8 Currying y partial application

Curry es un nombre atribuido a varias especias y platos generalmente picantes provenientes de varias regiones de Asia. No confundir con *Currying*, una técnica de programación para transformar una función de dos o más argumentos en una secuencia de funciones que reciben solamente un argumento.

Define la función `curry_twice(f)` para funciones binarias, tal que `curry_twice(f)(a)(b) = f(a,b)`.

```
function<int(int,int)> add = [](int a, int b) { return a + b; };
auto curried_add = curry_twice(add);
curried_add(2)(3); // >>> 5
```

Define la función `uncurry(f)` tal que `uncurry(f)(a,b) = f(a)(b)`. Puede asumir que solamente retornará funciones con dos parámetros.

```
uncurry(curried_add)(2, 3); // >>> 5
```

2.1.9 Ordenamiento

Define la función `sort_by(1st, comparator)` donde `comparator(a,b)` decide el orden.

```
sort_by(vector<int>{3, 1, 4, 2}, [](int a, int b) { return a < b; }); // >>> {1, 2, 3, 4}
sort_by(vector<int>{3, 1, 4, 2}, [](int a, int b) { return a > b; }); // >>> {4, 3, 2, 1}
```

2.1.10 Subcripciones

Hay tantas cosas ocurriendo en la granja que no puedes estar al tanto de todo, por lo que decides implementar tu propio sistema de subcripciones y notificaciones:

debes implementar los siguientes métodos en la clase `Publisher`:

- `subscribe(event_name, handler_name, handler)`: registra una función para un evento.
- `unsubscribe(event_name, handler_name)`: elimina una subcripción.
- `publish(event_name)`: ejecuta todos los `handler` suscritos a ese evento.

No se evaluará el orden en el que se emitan los eventos.

```
Publisher publisher;
bool is_choclo_listo = false;

auto set_choclo = [&]() { is_choclo_listo = true; };

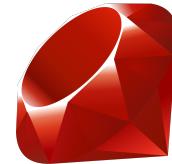
publisher.subscribe("choclo listo", "set_choclo", set_choclo);

publisher.publish("choclo listo");
is_choclo_listo; // >>> true

publisher.unsubscribe("choclo listo", "set_choclo");
```

2.2 Ruby (50%)

Ruby es un lenguaje de programación interpretado, reflexivo y orientado a objetos, creado por el programador japonés Yukihiro "Matz" Matsumoto en 1995. Combina una sintaxis inspirada en Python y Perl con características de programación orientada a objetos similares a Smalltalk. Comparte también funcionalidad con otros lenguajes de programación como Lisp, Lua, Dylan y CLU. Puedes instalarlo en ruby-lang.org.



2.2.1 Programación Orientada a Animales

Decides dejarle a los animales de la granja un regalo antes de irte: ¡su propio lenguaje para ellos! De esta manera, pueden usar clases para modelar establos, alimentos e invernaderos. Debes implementar un intérprete para un lenguaje orientado a objetos simple.

1. Debe soportar definición de clases, instanciamiento para objetos e invocación dinámica de métodos.
2. Debe ser posible heredar de otra clase, y reescribir los métodos definidos.
3. Contiene los tipos primitivos *int* y *string* junto con sus operaciones.
4. Debe implementar condicionales *if/else* y recursión.
5. Se debe acceder a las variables de instancia con `@`
6. Se retorna el último valor del body de un método.

Estructura del AST

1. ClassDef(name, parent, methods)
2. MethodDef(name, params, body)
3. MethodCall(receiver, method, args)
4. New(class_name, args)
5. Literal(value)
6. Variable(name)
7. Assignment(name, value)
8. If(condition, then_body, else_body)

```
interpreter = Interpreter.new

animal_class = ClassDef.new("Animal", nil, [
  MethodDef.new("initialize", ["name"], [
    Assignment.new("@name", Variable.new("name"))
  ]),
  MethodDef.new("speak", [], [
    Literal.new("El animal habla")
  ]),
  MethodDef.new("get_name", [], [
    Variable.new("@name")
  ])
])

interpreter.evaluate(animal_class)

dog_instance = interpreter.evaluate(New.new("Animal", [Literal.new("Perro")]))

speak_result = interpreter.evaluate(MethodCall.new(dog_instance, "speak", []))
# >>> "El animal habla"

name_result = interpreter.evaluate(MethodCall.new(dog_instance, "get_name", []))
# >>> "Perro"

dog_class = ClassDef.new("Dog", "Animal", [
  MethodDef.new("speak", [], [
    Literal.new("Guau!")
  ])
])

interpreter.evaluate(dog_class)

rex = interpreter.evaluate(New.new("Dog", [Literal.new("Rex")]))
speak_rex = interpreter.evaluate(MethodCall.new(rex, "speak", []))
# >>> "Guau!"

name_rex = interpreter.evaluate(MethodCall.new(rex, "get_name", []))
# >>> "Rex"
```

3 Entrega

3.1 Classroom

- *Link classroom:* <https://classroom.github.com/a/cvJdE4E0>

El repositorio tiene la siguiente estructura de archivos:

```
.gitignore
README.md
cpp/
ruby/
```

Los comandos para ejecutar los tests son:

```
ruby test.rb // en la carpeta ruby
make test // en la carpeta cpp
```

3.2 Evaluación

- **Importante:** recuerda ejecutar los tests que te entregaremos para asegurar que tu programa funciona.
- Cada parte de la tarea (C++ y Ruby) vale 50% del total de la evaluación.
- Debe modificar el archivo README.md con su nombre y número de alumno. Si no los agrega, arriesgas que tu tarea no se pueda revisar.
- Es posible que nos comuniquemos con usted si hay algún trozo de código que genere dudas o sospechas. No responder o no poder explicar el código puede resultar en descuentos a su calificación.
- Siempre y cuando se permita en el ejercicio, siéntanse libres de modificar y agregar clases y métodos.
- Se calificará el porcentaje de tests correctos: es decir que la nota de la tarea equivale a

$$\frac{\text{passing tests}}{\text{total tests}} * 6 + 1$$

- La **Fecha de Entrega:** es *Martes 26 de Mayo* hasta las 23:59.

3.3 Política de Atraso

Existe la posibilidad de entregar esta evaluación con hasta **2 días de atraso**. En ese caso, **la nota máxima a obtener se reduce** en 10 décimas por cada día de atraso. La fórmula para calcular la nota final con descuento es:

$$\text{nota_final} = \min(7.0 - (1.0 * \text{dias_de_atraso}), \text{nota_obtenida})$$

En otras palabras, la nota máxima para entregar al día siguiente es de 6.0, y la nota máxima a los dos días siguientes es de 5.0

4 Recomendaciones

4.1 Recomendaciones generales de parte de los ayudantes

- Está disponible un foro de **discussions** para las dudas de su tarea.
- Los lenguajes de la tarea estan asociados a sus carpetas y archivos de tests correspondiente.
- Los archivos tiene una estructura por defecto para las funciones que se te pide implementar.
- Son libres de editar los archivos con los que viene la tarea (a veces es necesario) siempre y cuando tengan cuidado que los tests sigan corriendo.
- Lee atentamente las notas resaltadas como **Importante** en el enunciado, ya que contiene restricciones sobre como puedes resolver el ejercicio.
- En caso de que falte información en el enunciado o los tests publicos para tomar decisiones respecto a la implementación, es libre de realizar suposiciones en como tratar con ciertos casos. Sin embargo, debes declarar en el archivo README.md que suposiciones realizo para cada parte.
- Si utilizas alguna herramienta como soporte para realizar su tarea, debes declararlo en el archivo README.md. Ten en cuenta las normas de integridad académica del curso.

- No "copien y peguen" una solución generada
- Su tarea será revisada manualmente por el cuerpo de ayudantes. Es por esto que se sugiere comentar su código y explicarlo de manera clara.

4.2 Recursos

4.2.1 C++

- **Tutorial:** <https://www.learncpp.com/>
- **Documentación:** <https://en.cppreference.com/>
- **Lambdas:** <https://en.cppreference.com/w/cpp/language/lambda>.
- **Templates:** <https://en.cppreference.com/cpp/language/templates>.
- **Functional Programming:** <https://en.cppreference.com/w/cpp/header/functional>.
- **C++ Debugger:** <https://godbolt.org/>.

4.2.2 Ruby

- **Tutorial:** <https://www.ruby-lang.org/en/documentation/quickstart/>
- **Documentación:** <https://ruby-doc.org/>
- **Unit testing:** <https://ruby-doc.org/stdlib-3.1.0/libdoc/test-unit/rdoc/Test/Unit.html>

5 Uso de inteligencia Artificial

Respecto al uso de herramientas generadoras de código. Estas están permitidas siempre y cuando se referencie correctamente cuándo es utilizado. Junto a lo anterior, se espera que toda respuesta entregada por una herramienta generadora de código o de IA pase por un proceso de análisis crítico y modificación antes de ser incluida en una evaluación. En caso de entregar una tarea donde se identifique que solo hubo un proceso de copiar la respuesta dada por una herramienta generadora de código o de IA, esto no será aceptado y será considerado como una falta a la integridad académica.